

Computer Science 2

CS2050

Queues



METROPOLITAN STATE UNIVERSITY™
OF DENVER

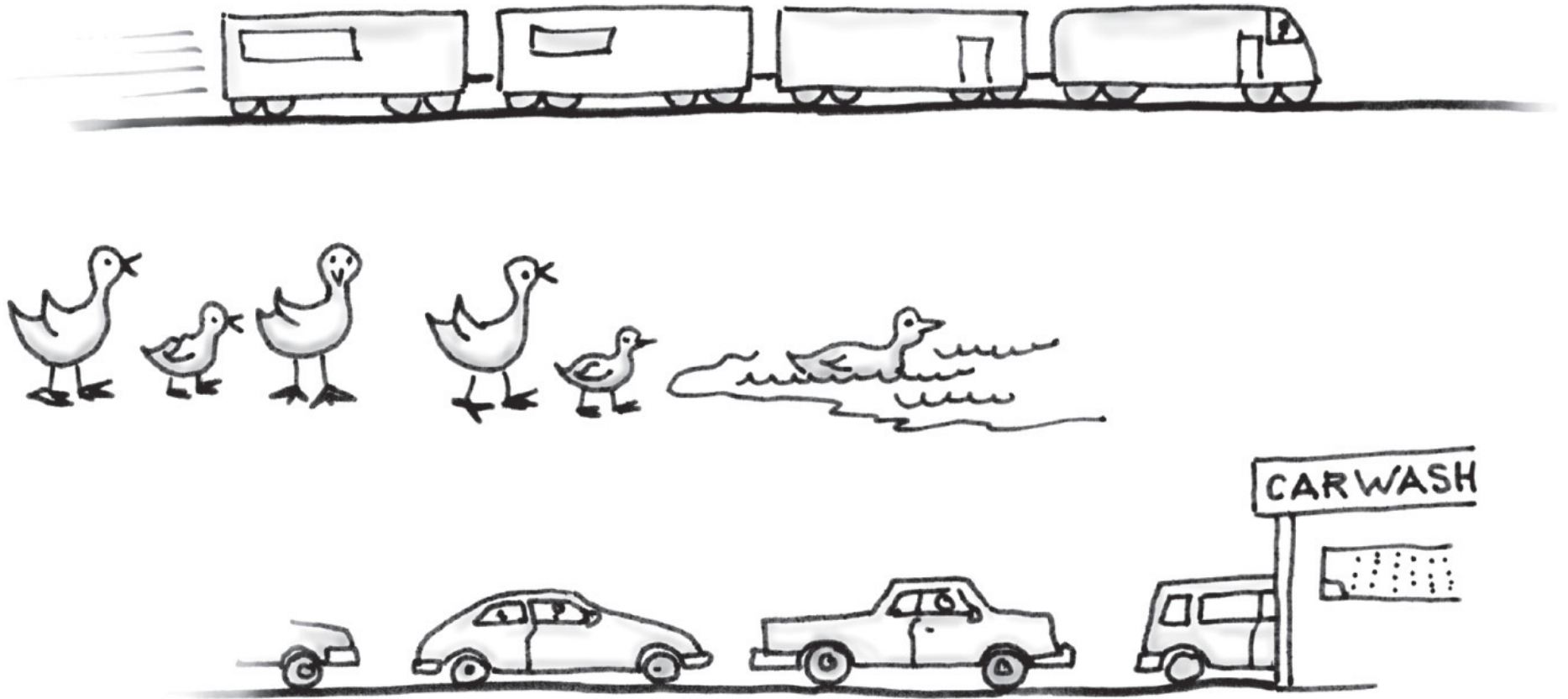


METROPOLITAN STATE UNIVERSITY™
OF DENVER

The ADT Queue

- A queue is another name for a waiting line
- Used within operating systems and to simulate real-world events
 - Come into play whenever processes or events must wait
- Entries organized first-in, first-out

The ADT Queue



© 2019 Pearson Education, Inc.

The ADT Queue

- Terminology
 - Item added first, or earliest, is at the front of the queue
 - Item added most recently is at the back of the queue
- Additions to a software queue must occur at its back

The ADT Queue

- Terminology
 - Item added first, or earliest, is at the front of the queue
 - Item added most recently is at the back of the queue
- Additions to a software queue must occur at its back
- Client can look at or remove only the entry at the front of the queue

The ADT Queue

- Data

- A collection of objects in chronological order and having the same data type

enqueue(newEntry)	+enqueue(newEntry: integer): void	Task: Adds a new entry to the back of the queue. Input: newEntry is the new entry. Output: None.
dequeue()	+dequeue(): T	Task: Removes and returns the entry at the front of the queue. Input: None. Output: Returns the queue's front entry. Throws an exception if the queue is empty before the operation.
getFront()	+getFront(): T	Task: Retrieves the queue's front entry without changing the queue in any way. Input: None. Output: Returns the queue's front entry. Throws an exception if the queue is empty.
isEmpty()	+isEmpty(): boolean	Task: Detects whether the queue is empty. Input: None. Output: Returns true if the queue is empty.
clear()	+clear(): void	Task: Removes all entries from the queue. Input: None. Output: None.

The ADT Queue

- Interface for the ADT queue

`QueueInterface.java`

FIGURE 7-2 The effect of operations on a queue of strings

(a) enqueue adds *Jada*

© 2019 Pearson Education, Inc.

Jada

(b) enqueue adds *Jess*

© 2019 Pearson Education, Inc.

Jada Jess

(c) enqueue adds *Jazmin*

© 2019 Pearson Education, Inc.

Jada Jess Jazmin

(d) enqueue adds *Jorge*

© 2019 Pearson Education, Inc.

Jada Jess Jazmin Jorge

(e) enqueue adds *Jamal*

© 2019 Pearson Education, Inc.

Jada Jess Jazmin Jorge Jamal

(f) dequeue retrieves and removes *Jada*

© 2019 Pearson Education, Inc.

Jada Jess Jazmin Jorge Jamal

(g) enqueue adds *Jerry*

© 2019 Pearson Education, Inc.

Jess Jazmin Jorge Jamal Jerry

(h) dequeue retrieves and removes *Jess*

© 2019 Pearson Education, Inc.

Jess Jazmin Jorge Jamal Jerry

Exercise 1

After the following nine statements execute, what string is at the front of the queue and what string is at the back?

```
QueueInterface<String> myQueue = new  
LinkedListQueue<>();  
myQueue.enqueue("Jada");  
myQueue.enqueue("Jess");  
myQueue.enqueue("Jazmin");  
myQueue.enqueue("Jorge");  
String name = myQueue.dequeue();  
myQueue.enqueue(name);  
myQueue.enqueue(myQueue.getFront());  
name = myQueue.dequeue();
```

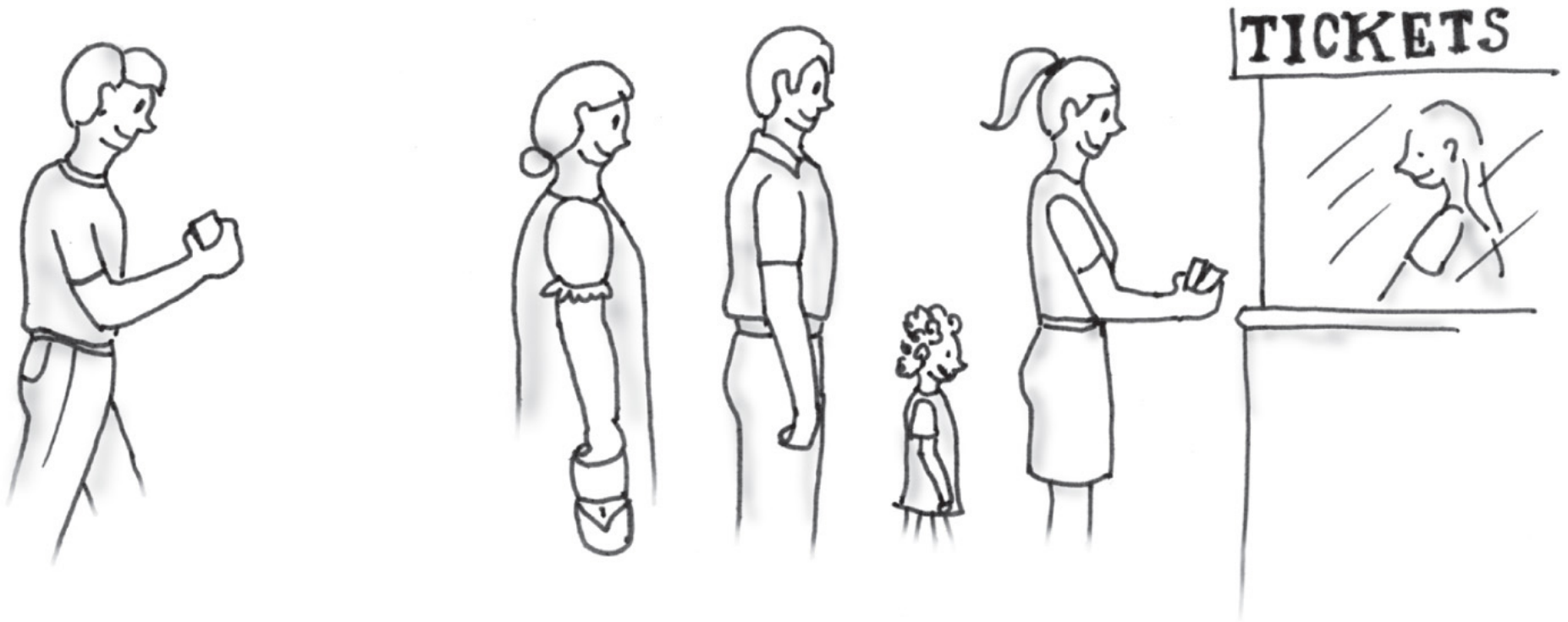
Exercise 1

After the following nine statements execute, what string is at the front of the queue and what string is at the back?

```
QueueInterface<String> myQueue = new  
LinkedQueue<>(); myQueue.enqueue("Jada");  
myQueue.enqueue("Jess");  
myQueue.enqueue("Jazmin");  
myQueue.enqueue("Jorge"); String name =  
myQueue.dequeue(); myQueue.enqueue(name);  
myQueue.enqueue(myQueue.getFront()); name =  
myQueue.dequeue();
```

Jazmin is at the front, Jess is at the back.

Simulating a Waiting Line



© 2019 Pearson Education, Inc.

Simulating a Waiting Line

- FIGURE 7-4 A CRC card for the class WaitLine

<i>WaitLine</i>
<i>Responsibilities</i>
<i>Simulate customers entering and leaving a</i>
<i>waiting line</i>
<i>Display number served, total wait time,</i>
<i>average wait time, and number left in line</i>
<i>Collaborations</i>
<i>Customer</i>



Simulating a Waiting Line

- FIGURE 7-5 A diagram of the classes WaitLine and Customer

1

WaitLine

line—a queue of customers
numberOfArrivals—number of customers
numberServed—number of customers actually served
totalTimeWaited—total time customers have waited

simulate(duration, arrivalProbability, maxTransactionTime)
displayResults()

*

Customer

arrivalTime
transactionTime
customerNumber

getArrivalTime()
getTransactionTime()
getCustomerNumber()



Simulating a Waiting Line

- Algorithm for simulate

Simulating a Waiting Line (Part 1)

Transaction time left: 5



Time: 0



Wait: 0

Customer 1 enters line with a 5-minute transaction.
Customer 1 begins service after waiting 0 minutes.

Transaction time left: 4



Time: 1



Customer 1 continues to be served.

Transaction time left: 3



Time: 2



Customer 1 continues to be served.
Customer 2 enters line with a 3-minute transaction.

Transaction time left: 2



Time: 3



Customer 1 continues to be served.

Transaction time left: 1



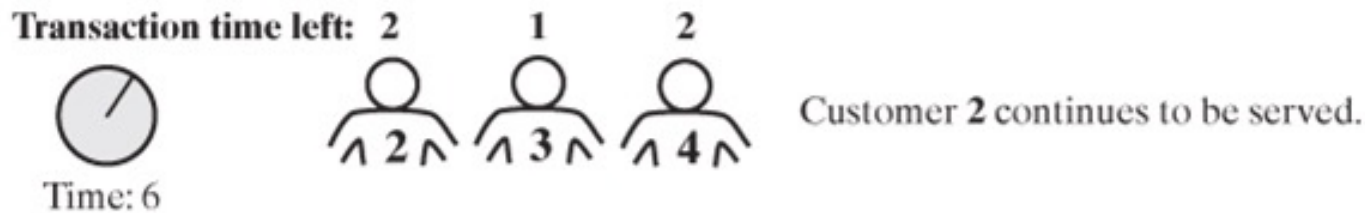
Time: 4



Customer 1 continues to be served.
Customer 3 enters line with a 1-minute transaction.



Simulating a Waiting Line (Part 2)



Exercise 2

Consider the simulation begun show above

a. At what time does Customer 4 finish and depart?

b. How long does Customer 5 wait before beginning the transaction?

Exercise 2

Consider the simulation begun show above

a. At what time does Customer 4 finish and depart?

11

b. How long does Customer 5 wait before beginning the transaction?

4

Java Class Library: The Interface Queue

Java Class Library: The Interface Queue

- Methods provided
 - add

Java Class Library: The Interface Queue

- Methods provided
 - add
 - offer

Java Class Library: The Interface Queue

- Methods provided
 - add
 - offer
 - remove

Java Class Library: The Interface Queue

- **Methods provided**
 - add
 - offer
 - remove
 - poll

Java Class Library: The Interface Queue

- **Methods provided**

- add
- offer
- remove
- poll
- element

Java Class Library: The Interface Queue

- **Methods provided**

- add
- offer
- remove
- poll
- element
- peek

Java Class Library: The Interface Queue

- **Methods provided**

- add
- offer
- remove
- poll
- element
- peek
- isEmpty

Java Class Library: The Interface Queue

- **Methods provided**

- add
- offer
- remove
- poll
- element
- peek
- isEmpty
- size

Java Class Library: The Interface Queue

- **Methods provided**

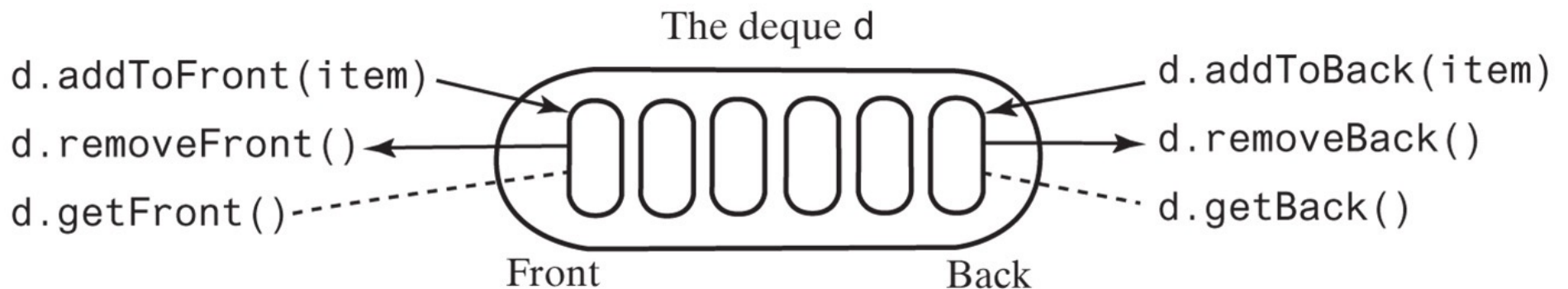
- add
- offer
- remove
- poll
- element
- peek
- isEmpty
- size
- clear

The ADT Deque

- A double ended queue
- Deque pronounced “deck”
- Has both queue-like operations and stack-like operations

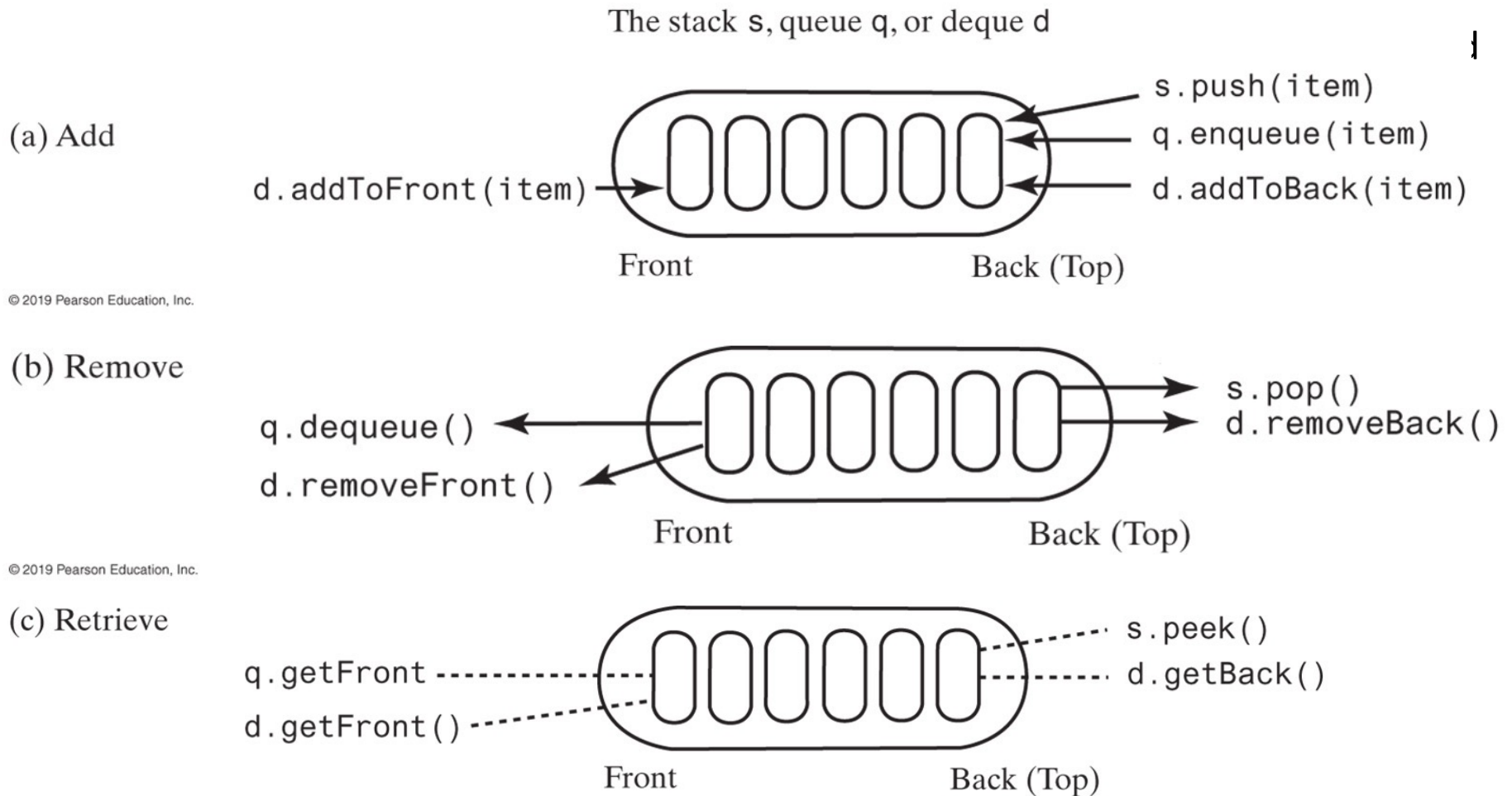
The ADT Deque

- FIGURE 7-10 An instance d of a deque



© 2019 Pearson Education, Inc.

FIGURE 7-11 A comparison of operations for a stack s, a queue q, and a deque d



© 2019 Pearson Education, Inc.

© 2019 Pearson Education, Inc.

© 2019 Pearson Education, Inc.



Exercise 3

After the following nine statements execute, what string is at the front of the deque and what string is at the back?

```
DequeInterface<String> myDeque = new  
LinkedDeque<>(); myDeque.addToFront("Jim");  
myDeque.addToBack("Jess");  
myDeque.addToFront("Jill");  
myDeque.addToBack("Jane"); String name =  
myDeque.getFront(); myDeque.addToBack(name);  
myDeque.removeFront();  
myDeque.addToFront(myDeque.removeBack());
```


Exercise 3

After the following nine statements execute, what string is at the front of the deque and what string is at the back?

```
DequeInterface<String> myDeque = new  
LinkedDeque<>(); myDeque.addToFront("Jim");  
myDeque.addToBack("Jess");  
myDeque.addToFront("Jill");  
myDeque.addToBack("Jane"); String name =  
myDeque.getFront(); myDeque.addToBack(name);  
myDeque.removeFront();  
myDeque.addToFront(myDeque.removeBack());
```

Jill is at the front, Jane is at the back

The ADT Deque

```
/** An interface for the ADT deque. */
public interface DequeInterface<T>
{
    /** Adds a new entry to the front/back of this deque.
        @param newEntry An object to be added. */
    public void addToFront(T newEntry);
    public void addToBack(T newEntry);

    /** Removes and returns the front/back entry of this deque.
        @return The object at the front/back of the deque.
        @throws EmptyQueueException if the deque is empty before the
            operation. */
    public T removeFront();
    public T removeBack();

    /** Retrieves the front/back entry of this deque.
        @return The object at the front/back of the deque.
        @throws EmptyQueueException if the deque is empty. */
    public T getFront();
    public T getBack();

    /** Detects whether this deque is empty.
        @return True if the deque is empty, or false otherwise. */
    public boolean isEmpty();

    /** Removes all entries from this deque. */
    public void clear();
} // end DequeInterface
```

Java Class Library: The Interface Deque

Java Class Library: The Interface Deque

- Methods provided
 - `addFirst`, `offerFirst`

Java Class Library: The Interface Deque

- **Methods provided**
 - `addFirst, offerFirst`
 - `addLast, offerLast`

Java Class Library: The Interface Deque

- **Methods provided**
 - `addFirst, offerFirst`
 - `addLast, offerLast`
 - `removeFirst, pollFirst`

Java Class Library: The Interface Deque

- **Methods provided**
 - `addFirst, offerFirst`
 - `addLast, offerLast`
 - `removeFirst, pollFirst`
 - `removeLast, pollLast`

Java Class Library: The Interface Deque

- **Methods provided**
 - `addFirst, offerFirst`
 - `addLast, offerLast`
 - `removeFirst, pollFirst`
 - `removeLast, pollLast`
 - `getFirst, peekFirst`

Java Class Library: The Interface Deque

- **Methods provided**
 - `addFirst, offerFirst`
 - `addLast, offerLast`
 - `removeFirst, pollFirst`
 - `removeLast, pollLast`
 - `getFirst, peekFirst`
 - `getLast, peekLast`

Java Class Library: The Interface Deque

- **Methods provided**
 - `addFirst, offerFirst`
 - `addLast, offerLast`
 - `removeFirst, pollFirst`
 - `removeLast, pollLast`
 - `getFirst, peekFirst`
 - `getLast, peekLast`
 - `isEmpty, clear, size`

Java Class Library: The Interface Deque

- **Methods provided**
 - `addFirst, offerFirst`
 - `addLast, offerLast`
 - `removeFirst, pollFirst`
 - `removeLast, pollLast`
 - `getFirst, peekFirst`
 - `getLast, peekLast`
 - `isEmpty, clear, size`
 - `push, pop`

Java Class Library: The Class `ArrayDeque`

- Implements the interface **`Deque`**
- Constructors provided
 - **`ArrayDeque()`**
 - **`ArrayDeque(int initialCapacity)`**

ADT Priority Queue

- Consider how a hospital assigns a priority to each patient that overrides time at which patient arrived.
- ADT priority queue organizes objects according to their priorities
- Definition of “priority” depends on nature of the items in the queue

ADT Priority Queue

```
/**An interface for the ADT priority queue.*/
public interface PriorityQueueInterface<T extends Comparable<? super T>>
{
    /** Adds a new entry to this priority queue.
        @param newEntry An object to be added. */
    public void add(T newEntry);

    /** Removes and returns the entry having the highest priority.
        @return Either the object having the highest priority or, if the
                priority queue is empty before the operation, null. */
    public T remove();

    /** Retrieves the entry having the highest priority.
        @return Either the object having the highest priority or, if the
                priority queue is empty, null. */
    public T peek();

    /** Detects whether this priority queue is empty.
        @return True if the priority queue is empty, or false otherwise. */
    public boolean isEmpty();

    /** Gets the size of this priority queue.
        @return The number of entries currently in the priority queue. */
    public int getSize();

    /** Removes all entries from this priority queue. */
    public void clear();
} // end PriorityQueueInterface
```



Excercise 4

After the following statements execute, what string is at the front of the priority queue and what string is at the back? Let the alphabetic order of the strings determine their priority. Note that "z" has a higher priority than "a".

```
PriorityQueueInterface<String> myPriorityQueue = new  
LinkedPriorityQueue<>();
```

```
myPriorityQueue.add("Jane");
```

```
myPriorityQueue.add("Jim");
```

```
myPriorityQueue.add("Jill");
```

```
String name = myPriorityQueue.remove();
```

```
myPriorityQueue.add(name);
```

```
myPriorityQueue.add("Jess");
```

Excercise 4

After the following statements execute, what string is at the front of the priority queue and what string is at the back? Let the alphabetic order of the strings determine their priority. Note that "z" has a higher priority than "a".

```
PriorityQueueInterface<String> myPriorityQueue = new  
LinkedPriorityQueue<>();
```

```
myPriorityQueue.add("Jane");
```

```
myPriorityQueue.add("Jim");
```

```
myPriorityQueue.add("Jill");
```

```
String name = myPriorityQueue.remove();
```

```
myPriorityQueue.add(name);
```

```
myPriorityQueue.add("Jess");
```

Jim is at the front, Jane is at the back.

Tracking Your Assignments

Assignment
course—the course code task—a description of the assignment date—the due date
getCourseCode() getTask() getDueDate() compareTo()

AssignmentLog
log—a priority queue of assignments
addProject(newAssignment) addProject(courseCode, task, dueDate) getNextProject() removeNextProject()



Tracking Your Assignments

```
public class AssignmentLog
{
    private PriorityQueueInterface<Assignment> log;

    public AssignmentLog()
    {
        log = new PriorityQueue<>();
    } // end constructor

    public void addProject(Assignment newAssignment)
    {
        log.add(newAssignment);
    } // end addProject

    public void addProject(String courseCode, String task, Date dueDate)
    {
        Assignment newAssignment = new Assignment(courseCode, task, dueDate);
        addProject(newAssignment);
    } // end addProject

    public Assignment getNextProject()
    {
        return log.peek();
    } // end getNextProject

    public Assignment removeNextProject()
    {
        return log.remove();
    } // end removeNextProject
} // end AssignmentLog
```

Java Class Library: The Class PriorityQueue

Java Class Library: The Class `PriorityQueue`

- Basic constructors and methods
- `PriorityQueue`
 - `add`

Java Class Library: The Class `PriorityQueue`

- Basic constructors and methods
- `PriorityQueue`
 - `add`
 - `offer`

Java Class Library: The Class PriorityQueue

- Basic constructors and methods
- PriorityQueue
 - add
 - offer
 - remove

Java Class Library: The Class `PriorityQueue`

- Basic constructors and methods
- `PriorityQueue`
 - `add`
 - `offer`
 - `remove`
 - `poll`

Java Class Library: The Class `PriorityQueue`

- Basic constructors and methods
- `PriorityQueue`
 - `add`
 - `offer`
 - `remove`
 - `poll`
 - `element`

Java Class Library: The Class `PriorityQueue`

- Basic constructors and methods
- `PriorityQueue`
 - `add`
 - `offer`
 - `remove`
 - `poll`
 - `element`
 - `peek`

Java Class Library: The Class PriorityQueue

- Basic constructors and methods
- PriorityQueue
 - add
 - offer
 - remove
 - poll
 - element
 - peek
 - isEmpty, clear, size

Queue, Dequeue, and Priority Queue Implementations